

Assignment 2 Discussion

CS 772

Vartul Bahuguna 24M0828
Atharv Shingewar 24M0834

Define Transliteration

Transliteration is writing words from one language using the alphabet/script of another language, focusing on representing the sounds rather than the meaning.

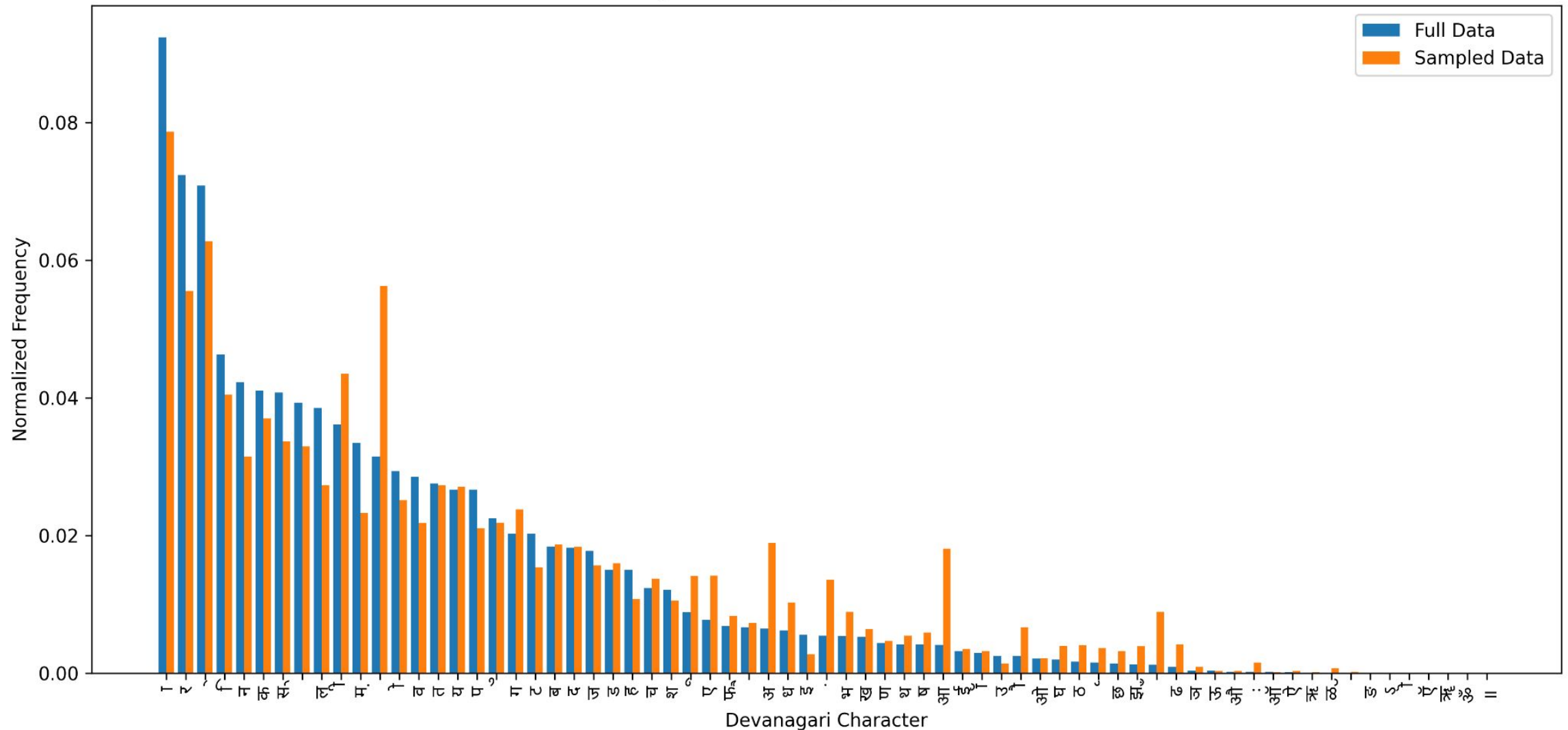
- Input: Atharv aur Vartul
- Output: अथर्व और वर्तुल

Data downloading and cleaning

- Number of training samples used = **100k**
- For sampling, earlier we tried with **hybrid sampling** method, that combines the **orthographic frequency**, **word length**, and **randomness** to select diverse examples. Words containing rarer orthographic units are prioritized, stratified by length, and supplemented with random samples to ensure balanced linguistic coverage and variety. But just using **random sampling** gave better results as the randomness brings more diversity to data.
- Sourced from [Aksharantar](#).
- We stripped other key value pairs of json entry and used only native word and english word key.

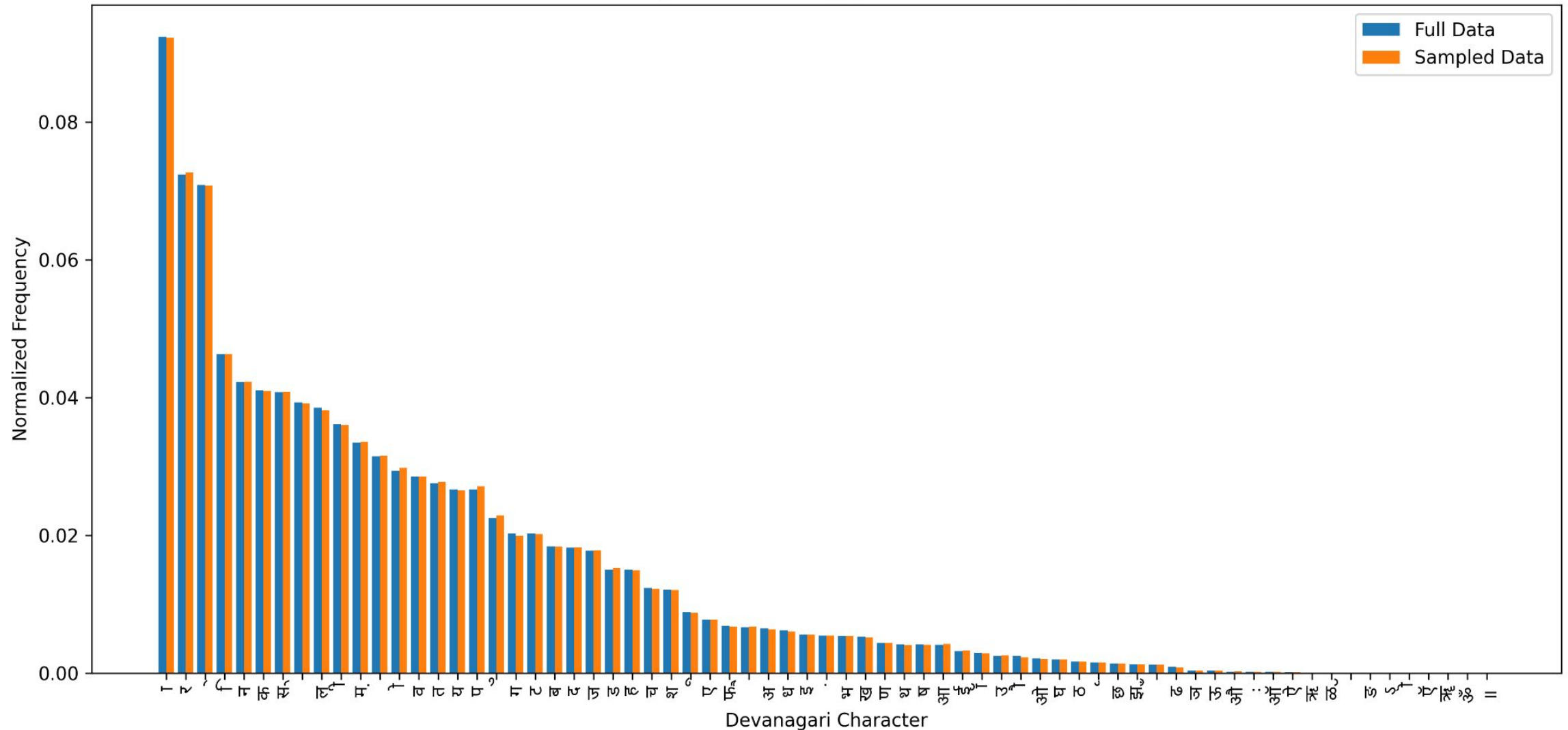
Sampling Comparison: **Hybrid** v/s Random

Top 100 Most Frequent Characters



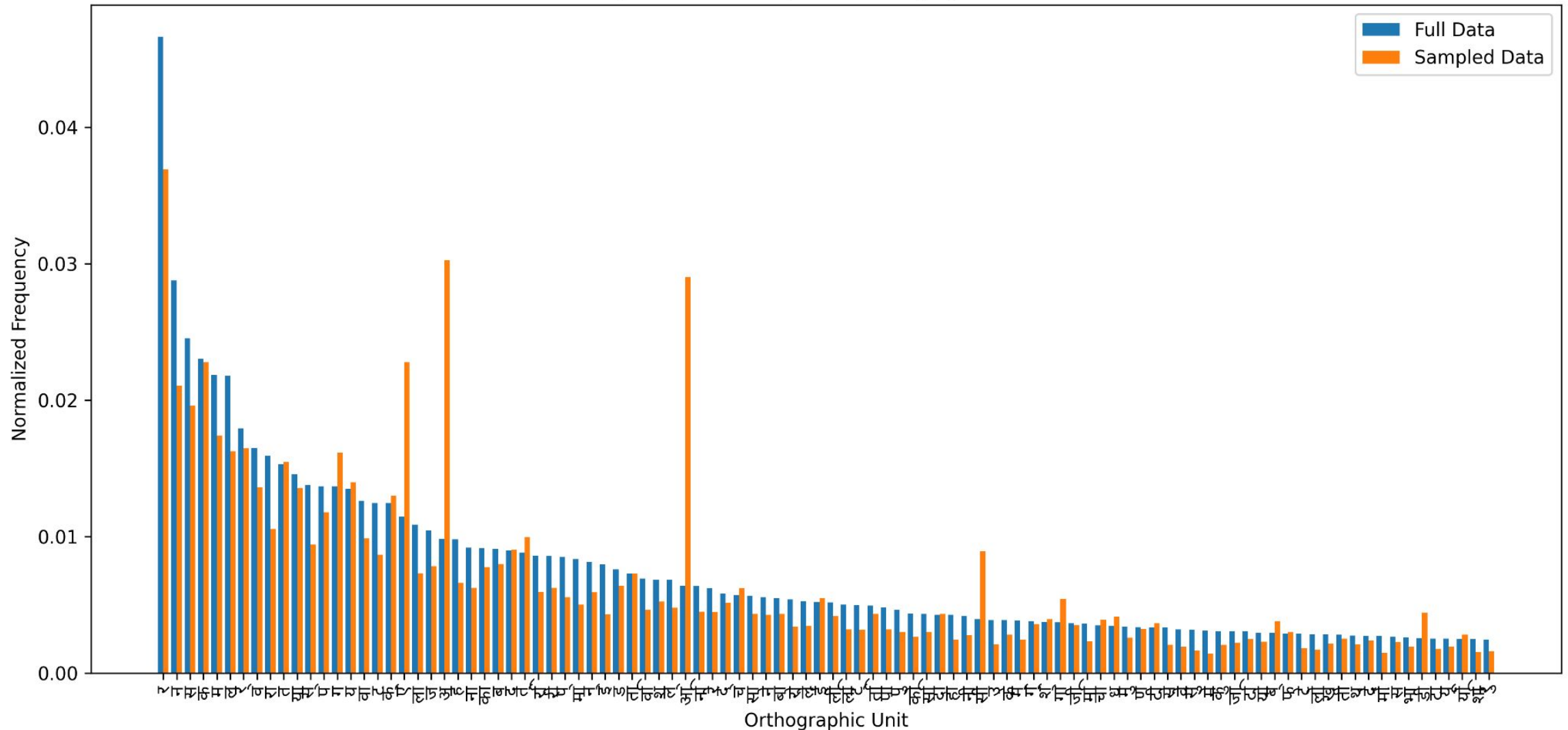
Sampling Comparison: Hybrid v/s **Random**

Top 100 Most Frequent Characters



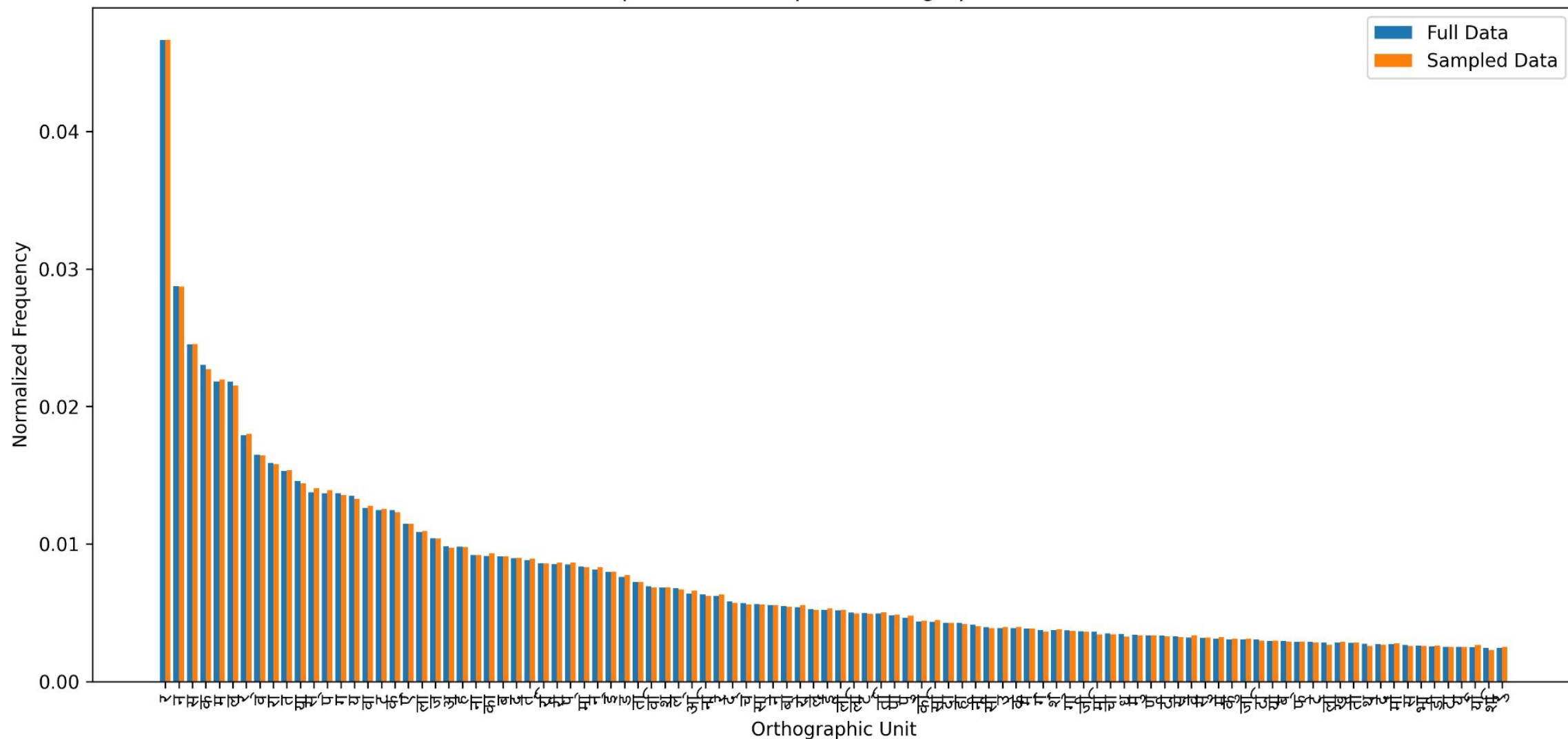
Sampling Comparison: **Hybrid** v/s Random

Top 100 Most Frequent Orthographic Units



Sampling Comparison: Hybrid v/s **Random**

Top 100 Most Frequent Orthographic Units



LSTM Based Transliteration - Hyperparameters used

Aspect	Description
Model Type	LSTM Sequence-to-Sequence
Embedding Dimension	128
Hidden Dimension	256
Number of Layers	2
Dropout	0.1
Batch Size	128
Learning Rate	0.001 (decayed on plateau)
Optimizer	Adam
Number of Epochs	50
Teacher Forcing Ratio	0.99 $\rightarrow$ 0.7 (decayed by 0.95)

LSTM Based Transliteration

Beam Width	Word Exact Accuracy(Test)	F score(Test)
1(Greedy)	0.4627	0.8880
3	0.4669	0.8889
5	0.4669	0.8889

LSTM Based Transliteration

- Beam search boosts the F score and accuracy marginally.
- Beam search helps a little for exact word matches, but after beam width 3, there is no significant gain.
- Increased inference time.

Transformer Based Transliteration - Hyperparameters Used

Aspect	Description
Model Type	Transformer (Seq2Seq)
Hidden Dimension (d_model)	256
Attention Heads	8
Encoder / Decoder Layers	2 each
Feedforward Dim	512
Dropout	0.1
Batch Size	64
Learning Rate	5e-4 (decayed on plateau)
Epochs	50
Teacher Forcing	0.99 $\rightarrow$ 0.7 (decayed gradually)

<https://docs.pytorch.org/docs/stable/generated/torch.nn.Transformer.html>

Transformer Based Transliteration

Beam Width	Word Exact Accuracy(Test)	F score(Test)
1(Greedy)	0.4764	0.8886
3	0.4782	0.8880
5	0.4781	0.8880

Transformer Based Transliteration

- Beam search provides a tiny boost in word-level accuracy ($\sim 0.18\%$), but no improvement in F score.
- No gain after beam width 3.
- Increased inference time.

LLM Based Transliteration

- Model Used: **GPT-4O-mini**
- Tested with various values of temperature and top_p values.
- Zero shot prompting used.

LLM Based Transliteration - Results(Test)

Temperature	Top_p	Word Exact Accuracy	F score
0.0	0.7	0.5144	0.8929
0.0.	0.85	0.5259	0.8980
0.0	1.0	0.5272	0.8984
0.4	0.7	0.5226	0.8981
0.4	0.85	0.5232	0.8970
0.4	1.0	0.5173	0.8968
0.8	0.7	0.5175	0.8968
0.8	0.85	0.5116	0.8951
0.8	1.0	0.4994	0.8882
1.2	0.7	0.5195	0.8966
1.2	0.85	0.5035	0.8916
1.2	1.0	0.4533	0.8642

LLM Based Transliteration

- Lower temperature (0–0.4) and Top-p $\approx$ 0.85–1.0 gives the best transliteration results.
- High temperature hurts exact match accuracy and F-score.

Error Analysis

1. Getting confused between sound of (त थ द ध न) and (ट ठ ड ढ ण)
(त थ द ध न) is pronounced with tongue touches teeth (dental)
(ट ठ ड ढ ण) is pronounced with tongue curled back (retroflex)
example
tamatar : तमातर (LSTM), तमातर (transformer)
damru: दामरू (LSTM), दमरू (Transformer)
dabba : दब्बा (LSTM), दब्बा (Transformer)
dhakkan : धक्कन (Transformer)
theek: थीक(LSTM)
thakaan: ठकां (Transformer)
ganana: गनाना (LSTM), गनाना (Transformer)

Error Analysis

2. getting confused between अ and आ.
 - a. kaam kam : काम काम (LSTM)
 - b. van : वान (Transformer)
 - c. arvind : आरविंद (Transformer)
 - d. bhaarat bharat : भारत भरत (Transformer) , भारत भारत (LSTM)
3. treating small words as abbreviation, example
 - e. to : टीओ (LSTM)
4. Hallucination:
 - f. ko: कोब (LSTM)

Project Proposal

Problem Statement: Investigate methods to convert a pre-trained small LM into an MoE model, train a lightweight gating mechanism, and measure trade-offs between efficiency, accuracy, and interpretability.

Motivation: LLM models consists of dense MLP layers that are heavy weighted. During inference not all of these weights give valuable contribution for next token generation, but since whole model has to be loaded, it requires a lot of GPU usage. Introducing MoE can help to activate only those Experts that contributes most to output for the next token.

Methodology: For this project we will go with a basic causal language modeling task. We plan to compare different MoE variants (like upcycling, virtual group, Drop-Upcycling) vs. dense models (gemma 270M) on inference latency, model size (parameters, memory footprint), and accuracy (or any baseline depending upon task) while measuring routing/load-balance behavior.

Paper referenced: [Upcycling Large Language Models into Mixture of Experts](#)
[Drop-Upcycling: Training Sparse Mixture of Experts with Partial Re-initialization](#)
[DeRS: Towards Extremely Efficient Upcycled Mixture-of-Experts Models](#)